

Robot Twin AI V1.0

软件架构设计文档

Robot Twin AI V1.0 软件架构设计文档

一、系统定位

Robot Twin AI V1.0 是面向具身智能研究的机器人数字孪生实验平台。

当前目标不是实现完整机器人自进化，而是建立第一个可扩展闭环：

真实机器人 → 数据采集 → 数字孪生 → AI分析 → 参数优化 → 真实机器人验证。

巡线小车只是第一个机器人实例，未来架构需要支持无人机、机械臂等设备。

二、总体软件架构

系统分为六个模块：

1. Embedded Firmware Layer（嵌入式层）

负责：

- 电机控制；
- 传感器采集；
- IMU数据读取；
- 通信协议；
- 状态上传。

输出统一机器人状态数据。

建议数据：

robot_id
timestamp
position
velocity
orientation

motor_state
battery
sensor_data
control_parameter

2. Communication Layer（通信层）

负责连接机器人与上位系统。

初期：
ESP无线通信。

后续：
支持MQTT、WebSocket、ROS2通信。

要求：

- 数据可靠传输；
- 时间同步；
- 异常重连；
- 数据校验。

3. Robot State Database（机器人数据层）

负责保存机器人长期运行数据。

保存：

- 每次运行记录；
- 参数变化；
- 故障记录；
- 优化历史。

数据库结构：

Robot表：
机器人基础信息。

Run表：
每次运行数据。

Parameter表：
控制参数变化。

Event表：
异常事件。

4. Digital Twin Engine（数字孪生引擎）

负责建立虚拟机器人。

包含：

Geometry Model：
机器人三维结构。

State Model：
位置、速度、姿态。

Behavior Model：
运动规律、电机响应、控制逻辑。

核心要求：

虚拟机器人状态必须与真实机器人同步。

5. AI Agent Layer（智能体层）

Agent定位：

机器人研发助手，而不是聊天机器人。

Agent能力：

工具1：
读取机器人历史数据。

工具2：
分析运行状态。

工具3：
调用数字孪生仿真。

工具4：
修改机器人参数。

工具5：
生成测试报告。

Agent工作流程：

发现问题
↓
读取数据
↓
分析原因
↓
提出方案
↓
仿真验证
↓
输出优化结果。

6. Optimization Layer（优化层）

负责实现机器人自主优化。

第一阶段：

人工确认AI建议。

第二阶段：

AI自动修改参数。

第三阶段：

AI自动生成代码并测试。

三、MCP工具设计

未来Agent通过MCP调用工程工具。

工具接口：

`read_robot_state()`

读取当前机器人状态。

`analyze_failure()`

分析运行失败原因。

`run_simulation()`

启动数字孪生测试。

`modify_parameter()`

修改控制参数。

`compile_firmware()`

编译固件。

`flash_device()`

烧录设备。

`generate_report()`

生成优化报告。

四、开发阶段规划

V1.0：

完成数据闭环。

验收：

真实机器人数据上传。

网页显示实时状态。

运行记录保存。

V1.5：

完成数字孪生同步。

验收：

虚拟机器人跟随真实机器人运动。

V2.0：

加入AI分析。

验收：

AI能够解释机器人运行问题。

V2.5：

加入自动优化。

验收：

AI提出参数优化方案，并经过仿真验证。

V3.0：

形成机器人自主优化闭环。

验收：

机器人发现问题 → AI优化 → 自动部署 → 性能提升。

五、Codex开发约束

开发时必须遵循：

1. 优先保证系统架构，不追求快速堆功能。
2. 所有模块必须接口化。
3. 数据格式必须统一。
4. Agent必须通过工具工作。
5. 所有自动修改必须经过验证。
6. 不允许把项目简化为普通网页控制小车。

六、最终目标

建立一个面向具身智能的机器人基础平台：

机器人拥有：

- 身份；
- 状态；
- 记忆；
- 数字模型；
- AI助手；
- 优化能力。

最终探索：

让机器人从被人工开发和调试的设备，逐渐成为能够理解自身、优化自身的智能系统。